

A Multi-Layer Workflow for Diagnosing Performance Portability in RAJA Applications

Befikir Bogale*, Ian Lumsden*, Stephanie Brink†, Olga Pearce†, Michela Taufer*

* University of Tennessee, Knoxville, TN, USA

{bbogale, ilumsden, mtaufer}@utk.edu

† Lawrence Livermore National Laboratory, Livermore, CA, USA

{brink2, olga}@llnl.gov

Abstract—Performance-portable frameworks like RAJA enable a single source to run on diverse HPC architectures, but productivity relies on the compiler removing abstraction overhead. When it does not, runtime data show where performance diverges, but not why. We present a multi-layer workflow that diagnoses abstraction-induced divergence by combining runtime measurements with compiler-visible evidence, including GPU resource usage, optimization remarks, and LLVM IR metrics. We apply this workflow to 55 RAJA Performance Suite kernels using HIP and OpenMP backends, three optimization levels, and three problem sizes (1,980 configurations). For selected kernels, the workflow distinguishes optimization-agnostic from optimization-sensitive behavior and links performance losses to the compiler’s excessive loop unrolling, register pressure, reduced occupancy, scratch spills, and control-flow complexity. These diagnostics guide source changes that restore native-level performance for four kernels and an optimization-specific transformation that eliminates compiler-induced register pressure in a fifth, providing guidance for understanding and improving performance portability beyond runtime analysis.

Index Terms—Performance Portability, RAJA, LLVM, Compiler Analysis, GPU, Performance Analysis, High Performance Computing

I. INTRODUCTION

Performance-portable frameworks such as RAJA [1] enable developers to maintain a single source implementation across heterogeneous CPU and GPU architectures. This portability improves software productivity by reducing duplicated backend-specific code, but its value depends on preserving performance relative to corresponding backend-native implementations. Achieving that goal requires the compiler to eliminate overhead introduced by mechanisms such as lambda encapsulation, policy-based dispatch, and high-level loop constructs. When this elimination is incomplete, semantically equivalent Native and performance-portable implementations can exhibit different runtime behavior for reasons that are not apparent from the source code. As a result, developers face the difficult task of diagnosing performance portability losses because the same abstractions that improve portability also obscure the underlying causes of performance degradation.

Prior work has shown that performance differences between portability frameworks and native implementations are common. Davis et al. [2] compare multiple GPU programming models—including CUDA, HIP, Kokkos, RAJA, OpenMP,

OpenACC, and SYCL—across proxy applications and production systems, demonstrating that programming model choice alone can lead to substantial performance variability. However, runtime measurements alone reveal where these differences occur but provide limited insight into why they arise. In practice, similar performance gaps can result from fundamentally different compiler behaviors, including additional control flow in hot loops, increased register pressure that reduces occupancy, or missed optimization opportunities. Distinguishing among these causes is essential because each requires a different strategy for diagnosing and improving performance portability.

This paper addresses this challenge by presenting a multi-layer workflow for diagnosing performance portability in applications built with portability abstractions. Our central observation is that performance divergence often originates from structural differences introduced during compilation, even when source-level implementations are semantically equivalent. To systematically identify and explain these differences, the workflow progressively combines runtime measurements with compiler-visible evidence collected across the compilation pipeline, including LLVM Intermediate Representation (IR) metrics, compiler optimization remarks, and GPU backend resource usage. Our multi-layer diagnostic workflow examines performance divergence across three complementary layers: the runtime layer, which identifies where Native and RAJA performance diverges; the compiler middle-end layer, which uses optimization remarks and LLVM IR metrics to identify structural causes; and the backend resource layer, which reveals hardware-resource consequences such as register pressure, occupancy, and spilling. These complementary sources of information enable developers to move beyond identifying where performance diverges to understanding why it diverges and how it can be improved.

Using the RAJA Performance Suite (RAJAPerf) [3], [4], we evaluate the proposed workflow by comparing Native and RAJA implementations of 55 kernels across HIP [5] and OpenMP [6] backends, three problem sizes, and three optimization levels, comprising 1,980 experimental configurations and 59,400 executions on LLNL’s Tuolumne system [7]. Our analysis shows that most statistically significant runtime differences are within 5%, indicating that portability

abstractions generally preserve performance. For the smaller subset of kernels exhibiting meaningful performance gaps, the workflow distinguishes persistent (optimization-agnostic) from optimization-sensitive behaviors and traces them to diagnosable compiler-level effects. We further identify hybrid kernel pairs that isolate the impact of individual abstraction features, providing controlled case studies that illustrate how the workflow guides the diagnosis of performance portability.

This paper makes the following contributions:

- We present a multi-layer workflow for diagnosing performance portability by progressively combining runtime measurements with compiler-visible evidence, including LLVM IR metrics, compiler optimization remarks, and GPU Kernel Resource Usage (KRU) metrics. The workflow systematically identifies the compiler behaviors responsible for abstraction-induced performance differences.
- We evaluate the workflow across a comprehensive experimental space spanning programming model, target backend, problem size, and optimization level. Using 55 RAJAPerf kernels, we identify three recurring categories of performance-portability behavior—optimization-agnostic, optimization-sensitive, and hybrid—that distinguish benign abstraction effects from those requiring developer intervention.
- We demonstrate that the workflow is actionable by guiding targeted optimizations that recover native-level performance. Compiler diagnostics identify source-level restructuring opportunities that reduce unnecessary control-flow complexity in four kernels, while exposing an optimization-sensitive case where disabling loop unrolling eliminates register pressure, scratch spilling, and the associated performance loss.

Runtime measurements are collected with Caliper [8] and integrated with static compiler information using Thicket [9]. To facilitate adoption and reproducibility, the diagnostic plugins and analysis scripts implementing the proposed workflow will be made publicly available with the publication artifact.

II. RAJA ABSTRACTION AND COMPILER DIAGNOSTICS

We study RAJA [1], a C++ performance portability library that separates application logic from backend-specific parallel execution models such as HIP and OpenMP. Developers express computation through loop-based interfaces such as `forall`, `kernel`, and `launch`, combined with execution policies that map iteration spaces onto hardware. Loop bodies are written as lambda functions that encapsulate per-iteration work independently of the target backend. These interfaces can introduce abstraction-induced overhead through lambda captures that encapsulate per-iteration state, execution policies that dispatch to backend-specific code paths, and indirection layers that map iteration spaces onto hardware threads. The cost of these abstractions depends on the compiler’s ability to eliminate them during optimization. When elimination is successful, the generated code closely matches a Native variant. When not,

residual abstraction in the IR can propagate to the allocation of backend resources and affect runtime performance.

Limits of Runtime-Only Evaluation. Runtime measurements identify where Native and RAJA variants differ in performance, but cannot explain the root causes because execution time collapses all compiler effects into a single observable value. Differences in execution time can stem from IR growth and control-flow restructuring, loop-structure changes, SSA-pressure effects on register allocation and occupancy, or missed optimization opportunities such as vectorization and unrolling. None of these internal changes are visible at runtime, making it impossible to determine whether a performance gap arises from structural complexity, missed optimizations, or hardware resource pressure. We look beyond runtime into evidence that the compiler leaves at every pipeline stage.

Compilation Pipeline and Data Collection. We analyze the compilation pipeline from source code through LLVM IR optimization to backend hardware resource allocation. Static compiler information is collected per function using two custom LLVM [10] pass plugins integrated with the Caliper [8] profiling framework. The first plugin, *extraction*, records AMD GPU kernel resource metrics, compiler optimization remarks, and LLVM IR structure during compilation. It does not modify the compilation pipeline, so the resulting binary is used directly for runtime measurement. The second plugin, *association*, maps Caliper runtime regions to their compiled functions, such as HIP kernels and OpenMP outlined functions, so that function-level static metrics can be attached to runtime profile regions. Because the association plugin instruments the pipeline and can perturb both compiler and runtime behavior, we use two separate compilation flows. Runtime measurements and static metrics come from the extraction build; region-to-function mappings come from the association build. Thicket [9] then combines both outputs into a unified dataset for cross-layer analysis.

III. THE RAJA PERFORMANCE SUITE

We use the RAJA Performance Suite (RAJAPerf) [3], [4] as our experimental testbed. RAJAPerf is a curated set of benchmarks built to assess RAJA’s performance portability claims. It offers a set of loop-oriented computational kernels, each provided in several variants that implement the same computation via the RAJA abstraction layer, including Native and RAJA versions.

RAJAPerf is well-suited for this study because its kernels span a broad set of computational patterns (e.g., stencil computations, dense linear algebra, and other loop-dominated structures), demonstrating cases where compilers eliminate abstraction overhead, and cases where they do not. Equally important, Native and RAJA variants share a semantically equivalent algorithmic specification and are structured to be directly comparable across programming models.

Table I lists the full set of 55 considered kernels.

We design an experimental search space to isolate the conditions under which abstraction-induced performance differences

TABLE I: The 55 RAJAPerf kernels evaluated in this study.

Cat.	Kernel	Cat.	Kernel
Alg.	MEMCPY MEMSET		DIFF_PREDICT EOS
	CONVECTION3DPA DEL_DOT_VEC_2D DIFFUSION3DPA EDGE3D ENERGY FIR LTIMES LTIMES_NOVIEW	Lcals	FIRST_DIFF FIRST_SUM GEN_LIN_RECUR HYDRO_1D HYDRO_2D INT_PREDICT PLANCKIAN TRIDIAG_ELIM
Apps	MASS3DEA MASS3DPA MATVEC_3D_STENCIL NODAL_ACCUMULATION_3D PRESSURE VOL3D ZONAL_ACCUMULATION_3D	Poly.	2MM 3MM ADI ATAX FDTD_2D FLOYD_WARSHALL GEMM
	COPY8 DAXPY DAXPY_ATOMIC IF_QUAD INDEXLIST_3LOOP INIT3		GEMVER GESUMMV HEAT_3D JACOBI_1D JACOBI_2D MVT
Basic	INIT_VIEW1D INIT_VIEW1D_OFFSET MULADDSUB NESTED_INIT PI_ATOMIC	Stream	ADD COPY MUL TRIAD

TABLE II: The four axes of the experimental search space.

Axis	Rationale
Programming model	Compare Native against RAJA variants of the same kernel to isolate the effect of expressing computation through the portability abstraction layer.
Target backend	Evaluate HIP (GPU) and OpenMP (CPU) to distinguish architecture-specific effects from those that appear across backends.
Problem size	Sweep 8M, 32M, and 128M elements, spanning node underutilization, saturation, and overutilization, to separate utilization artifacts from persistent performance differences.
Optimization level	Evaluate -O1, -O2, and -O3 to distinguish persistent abstraction effects from those that appear or disappear under more aggressive optimization.

arise. We vary four factors that characterize the compilation and execution context: programming model (Native and RAJA), target backend (HIP and OpenMP), problem size (8M, 32M, and 128M elements), and optimization level (-O1, -O2, and -O3), as summarized in Table II.

All runs are conducted on LLNL’s Tuolumne system using the ROCm 6.4.0 [7] compiler toolchain on a single MI300A node. For HIP, we use four GPUs with one MPI rank per GPU; for OpenMP, we use a single rank with 84 CPU cores. Each configuration is repeated 30 times to support robust statistical analysis. The total experimental space spans $N_{\text{cfg}} = 55 \times 2 \times 2 \times 3 \times 3 = 1,980$ configurations and $N_{\text{runs}} = 1,980 \times 30 = 59,400$ kernel executions.

Diagnostic Metrics. We combine one runtime metric with three categories of static analysis metrics: KRU, compiler

TABLE III: Static analysis metrics organized by layers in the multi-layer workflow.

Type	Category	Metric	Description
Backend	Resources	VGPRs	Number of vector registers allocated per thread (max)
Backend	Resources	SGPRs	Number of scalar registers allocated per wavefront (max)
Backend	Resources	AGPRs	Number of accumulator registers allocated per thread (max)
Backend	Resources	Scratch	Scratch memory allocated per thread (bytes) (max)
Backend	Resources	LDS	Shared (LDS) memory allocated per workgroup (bytes) (max)
Backend	Resources	VGPRSpill	Number of VGPR values spilled to scratch memory (sum)
Backend	Resources	SGPRSpill	Number of SGPR values spilled (sum)
Backend	Resources	Occupancy	Maximum number of wavefronts per SIMD (min)
IR	Remark	loop-unroll	Passed loop unrolling opt.
IR	Remark	loop-vectorize	Passed loop vectorization opt.
IR	Remark	slp-vectorizer	Passed SLP vectorization opt.
IR	Control	BBs	Total num of basic blocks
IR	Control	BrInsts	Total num of branch instructions
IR	Control	BrEdges	Total num of control-flow edges
IR	Control	PHIs	Total num of PHI instructions
IR	Control	Loops	Total num of loops
IR	Control	LoopDepth	Maximum loop nesting depth
IR	Control	VecBodies	Num of loops whose bodies are in vectorized form
IR	Control	TailLoops	Scalar remainder (tail) loops
IR	Control	LIBranch	Loop-invariant branch conditions within loops
IR	Control	VerGuards	Patterns enabling loop versioning guards
IR	Control	IL_BB	Basic blocks within inner loops
IR	Control	IL_Br	Branch instructions within inner loops
IR	Control	IL_PHIs	PHI instructions within inner loops
IR	Control	LiveSSA	Maximum simultaneously live SSA values
IR	Memory	Loads	Total num of load instructions
IR	Memory	AS4Loads	Loads from address space 4 (constant memory)
IR	Memory	NonAS4Ld	Loads from non-constant memory spaces
IR	Memory	Stores	Total num of store instructions
IR	Memory	LICMLoads	Loads promoted via loop-invariant code motion
IR	Memory	IL_Loads	Load instructions within inner loops
IR	Memory	IL_Alloca	Inner-loop accesses from stack allocations
IR	Memory	MemcpyAlc	Memory copy operations targeting stack allocations
IR	Compute	IntOp	Int arithmetic instructions
IR	Compute	FPOp	FP arithmetic instructions
IR	Compute	GEPS	Address computation (GetElementPtr) instructions
IR	Compute	IL_IntOp	Int arithmetic in inner loops
IR	Compute	IL_FPOp	Float arithmetic in inner loops
IR	Compute	IL_GEPS	Address computation (GetElementPtr) in inner loops
IR	Compute	RedPHIs	PHI nodes representing reduction recurrences

remarks, and LLVM IR metrics. Runtime metrics show where Native and RAJA differ in performance; static analysis metrics indicate which compiler-level factors contribute. Table III lists all static analysis metrics. We measure **runtime** as wall-clock time per kernel region using Caliper, computing the mean and Native-RAJA percent difference across 30 runs per configuration. We use **KRU** metrics to capture the hardware-level consequences of compilation, identifying when performance differences stem from register pressure, spilling, or occupancy loss; they are available for HIP kernels only, so our OpenMP analysis relies on compiler remarks and LLVM IR metrics. We examine **compiler remarks** to identify which optimizations were applied and where the two variants diverge in applied transformations. We use **LLVM IR metrics** to characterize the structural footprint of optimized IR across three subcategories: *Control-flow* describes how

computation is organized, *Memory* describes data access patterns, and *Compute* describes instruction-level work.

IV. STATISTICAL AND PRACTICAL ANALYSIS

A. Filtering for Statistical Significance

Performance-portable abstractions such as RAJA aim to match native performance, but this depends on the compiler’s ability to eliminate abstraction overhead and expose equivalent optimization opportunities. We begin at the runtime level by identifying where and under what conditions Native and RAJA variants differ. To accomplish this, we evaluate RAJAPerf across the experimental search space defined in Table II.

For each configuration, we perform a statistical significance test comparing Native and RAJA runtimes using the Mann–Whitney U test [11] (i.e., a non-parametric test that compares two distributions without assuming normality), with Holm–Bonferroni [12] correction applied jointly to the 990 Native–RAJA hypothesis tests. Figure 1 presents the results as a heatmap over all 55 kernels considered. We observe two key aspects. First, performance divergence is not uniform across kernels and configurations. On HIP, fewer than 10 of 55 kernels show statistically significant differences, and when differences do arise, they tend to persist across problem sizes and optimization levels. On OpenMP, a larger number of kernels show statistically significant differences, but the pattern is less consistent across configurations. Second, the set of statistically significant kernels changes with problem size. As the problem size increases to 128M, the number of significant kernels on OpenMP decreases, suggesting that some differences at smaller sizes reflect node underutilization, where the problem is too small to fully occupy the hardware, rather than inherent abstraction behavior.

Finding 1: HIP and OpenMP diverge differently: HIP exhibits few but persistent divergences; OpenMP exhibits many but configuration-dependent divergences.

B. Filtering for Practical Significance

To reflect realistic execution conditions, we focus on the 32M problem size, where hardware resources are fully utilized and runtime variability is minimal. Figure 2 shows the kernels that exhibit statistically significant differences at 32M for at least one optimization level. Across both target backends, most kernels exhibit small differences. On HIP, most percent differences fall within $\pm 5\%$. One exception is *MASS3DEA*, which shows extreme degradation at O2 and O3, examined in detail in Section V. OpenMP yields more statistically significant kernels, but with similarly small magnitudes in most cases. Thus, we distinguish statistical significance from practical significance and introduce an absolute difference threshold of $>5\%$ as the criterion for deeper analysis.

Finding 2: Most statistically significant runtime differences between Native and RAJA are negligible (within 5%).

We also allow one exception to the 5% threshold: a kernel below this threshold is kept if it forms a structurally or semantically similar pair with another kernel. Thus, *GESUMMV* (0.8% on HIP) is retained as a representative Polybench kernel with only negligible difference, allowing comparison with other Polybench kernels that show much larger gaps. By the same reasoning, we keep the structurally corresponding OpenMP pairs *LTIMES/LTIMES_NOVIEW* and *INIT_VIEWID/INIT_VIEWID_OFFSET*.

We identify three recurring behavioral patterns:

Optimization-agnostic kernels show consistent sign and magnitude differences across all three optimization levels, indicating a persistent structural cause that optimization cannot fix. On HIP, these are *CONVECTION3DPA*, *ADI*, *ATAX*, *GEMVER*, *GESUMMV*, and *MVT*; on OpenMP, they are *MEMSET* and *FIR*.

Optimization-sensitive kernels show divergences whose sign or magnitude varies significantly across optimization levels, indicating that these differences stem from specific compiler transformations rather than fixed structural properties. On HIP, *MASS3DEA* and *EDGE3D* are in this category; on OpenMP, this behavior appears in *LTIMES*, *LTIMES_NOVIEW*, *INIT_VIEWID*, *INIT_VIEWID_OFFSET*, *NESTED_INIT*, and *3MM*.

Hybrid pairs are structurally similar kernels that differ only in Native–RAJA behavior, enabling controlled comparisons that isolate a single abstraction feature. On HIP, *GEMVER* and *GESUMMV* form such a pair; on OpenMP, *LTIMES* vs. *LTIMES_NOVIEW* isolates View indirection, and *INIT_VIEWID* vs. *INIT_VIEWID_OFFSET* isolates an index offset.

Finding 3: Kernels that diverge fall into three structural categories: optimization-agnostic, optimization-sensitive, and hybrid pairs.

Mystery 1: Runtime measurements reveal where Native and RAJA diverge but collapse all compiler effects into a single number. For each behavioral pattern, which layers of the compilation pipeline carry the signal that explains the gap?

V. BACKEND RESOURCE DIVERGENCE ANALYSIS

A. Kernel Resource Usage Metrics

To answer Mystery 1, we first examine the HIP target through Kernel Resource Usage (KRU) metrics at the back-end of the compilation pipeline. KRU metrics expose how compiled kernels utilize hardware resources, including vector general-purpose registers (VGPRs), scalar general-purpose

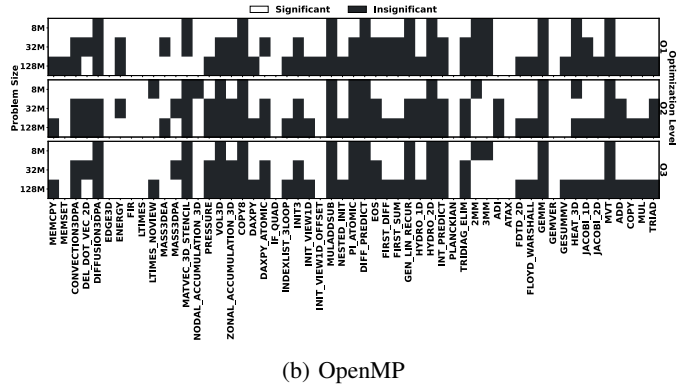
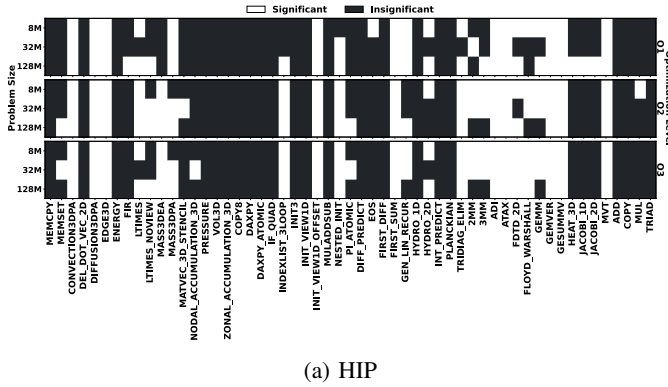


Fig. 1: Statistical significance of runtime differences between Native and RAJA on the HIP (a) and OpenMP (b) backends. The x-axis shows all 55 kernels; the y-axis shows the three problem sizes (8M, 32M, 128M) within each optimization level (O1, O2, O3). Black cells mark statistically significant differences (Mann–Whitney U, Holm–Bonferroni corrected, $p < 0.05$); white cells mark no significant differences.

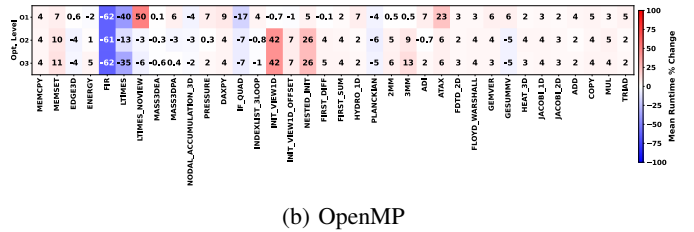
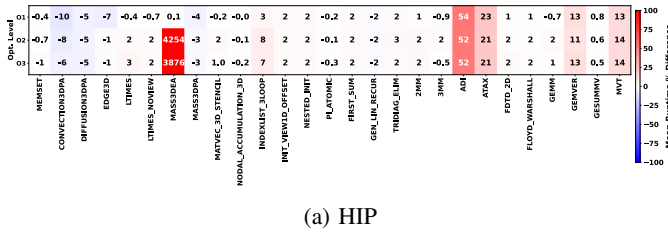


Fig. 2: Mean percent runtime difference, $(\text{RAJA} - \text{Native}) / \text{Native}$, at 32M for kernels with at least one statistically significant difference across optimization levels, on HIP (a) and OpenMP (b). The x-axis lists kernels; the y-axis shows optimization levels O1, O2, and O3. Positive values (red) mean RAJA is slower than Native; negative values (blue) mean RAJA is faster.

registers (SGPRs), accumulator registers (AGPRs), local data share (LDS), and scratch memory. On modern GPUs, runtime performance can be highly sensitive to resource usage, as these resources can constrain parallelism [7]. Figure 3 summarizes these metrics for the eight HIP kernels selected in Section IV-B.

By comparing Native and RAJA across these resource dimensions, we identify where resource usage diverges and whether those differences align with the runtime behavior observed in Section IV.

B. Occupancy Loss

The first diagnostic pattern concerns occupancy. Occupancy, defined as the number of active wavefronts per SIMD unit, is bounded by register and LDS availability. Higher occupancy improves latency hiding by enabling the GPU to schedule other wavefronts while one is stalled, so reductions can directly degrade performance. In our analysis, only *MASS3DEA* exhibits an occupancy divergence between Native and RAJA. At O1, both variants achieve the same occupancy, corresponding with a negligible runtime difference (approximately 0.1%). However, in O2 and O3, RAJA occupancy drops to one wavefront per SIMD, while Native maintains higher occupancy at two wavefronts per SIMD.

This reduction is driven by an increase in VGPR pressure in RAJA. At O2 and O3, RAJA uses 256 VGPRs, reaching

the architectural limit, whereas Native uses 183 VGPRs. Once the VGPR limit is saturated, the compiler attempts to alleviate pressure through spilling. This initially manifests as spilling into AGPRs, with RAJA using 256 AGPRs while Native uses none. However, AGPR spilling is insufficient to contain the excess register demand. After exhausting both VGPR and AGPR capacity, RAJA spills to expensive scratch memory, which resides in global memory. This results in substantial scratch usage — 2,888 bytes at O2 and 3,220 bytes at O3. The combined effect is twofold. First, elevated VGPR usage reduces occupancy, limiting the GPU’s ability to hide latency. Second, spilling to scratch memory introduces high-latency global memory accesses. Together, occupancy loss and spill overhead reinforce the performance degradation observed in RAJA at higher optimization levels.

Finding 4: Performance can degrade sharply when register pressure crosses architectural thresholds, leading to occupancy loss and spilling to scratch memory.

C. Scratch Memory to Register Tradeoff

A second diagnostic pattern appears when Native and RAJA have the same occupancy, but differ in how they manage resource usage. In this case, KRU metrics remain useful

		VGPRs		SGPRs		AGPRs		Scratch		LDS		VGPR Spill		SGPR Spill		Occupancy		
		N	R	N	R	N	R	N	R	N	R	N	R	N	R	N	R	
CONVECTION3DPA		42	26	43	20	0	0	0	0	3072	3072	0	0	0	0	6	6	O1
		58	36	54	26	0	0	0	0	3072	3072	0	0	0	0	6	6	O2
		58	36	56	26	0	0	0	0	3072	3072	0	0	0	0	6	6	O3
EDGE3D		256	256	66	66	76	160	1552	1360	0	0	0	0	0	0	1	1	O1
		256	256	74	58	160	158	0	0	0	0	14	14	0	0	1	1	O2
		256	256	74	58	160	158	0	0	0	0	14	14	0	0	1	1	O3
MASS3DEA		30	24	53	30	0	0	0	0	1168	1168	0	0	0	0	8	8	O1
		183	256	45	25	0	256	0	2888	1168	1168	0	895	0	0	2	1	O2
		183	256	45	25	0	256	0	3220	1168	1168	0	988	0	0	2	1	O3
ADI		38	26	42	46	0	0	0	0	0	0	0	0	0	0	8	8	O1
		38	26	44	52	0	0	0	0	0	0	0	0	0	0	8	8	O2
		38	26	44	52	0	0	0	0	0	0	0	0	0	0	8	8	O3
ATAX		12	14	16	26	0	0	0	0	0	0	0	0	0	0	8	8	O1
		12	14	18	26	0	0	0	0	0	0	0	0	0	0	8	8	O2
		12	14	18	26	0	0	0	0	0	0	0	0	0	0	8	8	O3
GEMVER		14	14	22	28	0	0	0	0	0	0	0	0	0	0	8	8	O1
		14	14	22	28	0	0	0	0	0	0	0	0	0	0	8	8	O2
		14	14	22	28	0	0	0	0	0	0	0	0	0	0	8	8	O3
GESUMMV		14	16	18	28	0	0	0	0	0	0	0	0	0	0	8	8	O1
		14	16	26	28	0	0	0	0	0	0	0	0	0	0	8	8	O2
		14	16	26	28	0	0	0	0	0	0	0	0	0	0	8	8	O3
MVT		10	14	16	22	0	0	0	0	0	0	0	0	0	0	8	8	O1
		10	14	18	24	0	0	0	0	0	0	0	0	0	0	8	8	O2
		10	14	18	24	0	0	0	0	0	0	0	0	0	0	8	8	O3

Fig. 3: KRU metrics for the eight HIP kernels at O1, O2, and O3, showing raw values for Native (N) and RAJA (R). Red cells indicate RAJA uses more of that resource; blue indicates less; white indicates no difference.

because they show whether the compiler keeps values in on-chip register resources or spills them to scratch memory. The key diagnostic question is therefore not whether resource usage reduces concurrency, as in Section V-B, but how each variant manages resources once occupancy is already fixed.

EDGE3D exhibits this pattern and contrasts with the behavior in Section V-B. Across all optimization levels, both Native and RAJA use 256 VGPRs, fixing occupancy at one wave per SIMD. Thus, the observed runtime differences cannot be attributed to differences in available wavefront concurrency. Instead, the relevant distinction is how each variant allocates values after VGPR usage has saturated the occupancy limit. At O1, RAJA uses more AGPRs than Native (160 vs. 76) but less scratch memory (1,360 bytes vs. 1,552 bytes). This indicates that RAJA retains more values in on-chip register storage, whereas Native uses scratch memory. Because scratch memory is more expensive, this difference provides a compiler-level insight for why RAJA is approximately 7% faster at O1 despite having the same occupancy as Native. At O2 and O3, this difference disappears. Both variants eliminate scratch usage and converge to similar AGPR usage, and the runtime

difference correspondingly falls to at most 1%. *EDGE3D* therefore shows that KRU metrics remain diagnostic even when occupancy is unchanged.

Finding 5: With fixed occupancy, the compiler can introduce performance differences through how it allocates values across storage tiers, particularly between on-chip registers and scratch memory.

D. Register Differences

A third diagnostic pattern occurs when neither occupancy nor scratch usage changes. Here, the most visible backend differences are variations in register usage, specifically VGPRs and SGPRs. This pattern appears in *CONVECTION3DPA* and the Polybench kernels *ADI*, *ATAX*, *GEMVER*, *GESUMMV*, and *MVT*. In *CONVECTION3DPA*, RAJA uses fewer VGPRs and SGPRs, which coincides with better RAJA performance. In contrast, for the Polybench kernels, RAJA uses more VGPRs and SGPRs, which is consistent with its slower performance. These observations suggest that even when occupancy is

unchanged, differences in register usage can still influence performance. *GESUMMV* is an exception to this trend. Although it exhibits higher VGPR and SGPR usage in RAJA, consistent with the other Polybench kernels, it does not show a practically significant performance difference between Native and RAJA. This shows that register usage alone is not the sole predictor of performance and that other compiler-visible factors must also contribute.

Finding 6: When occupancy and scratch usage remain unchanged, register usage differences alone provide limited explanatory power, indicating that additional compiler-visible structure must be considered.

Mystery 2: KRU metrics can highlight the influence of resource usage on performance divergence when register pressure exceeds hardware thresholds, but leave a residual gap for kernels in which occupancy, spill, and scratch usage are identical across Native and RAJA. For these kernels, what structural differences in the LLVM middle-end account for the observed runtime differences?

VI. LLVM IR LEVEL ANALYSIS

A. Changes in Successful Optimizations

To address Mystery 2, we move upstream to the LLVM middle-end, where optimization passes transform the IR prior

to backend resource allocation. This is also the first diagnostic layer for OpenMP, where no KRU-equivalent exists.

As a first step at this level, we examine differences in *passed* compiler optimization remarks, as shown in Figure 4. We focus on loop unrolling, loop vectorization, and SLP vectorization remarks available at O2 and O3.

We use these differences to identify where the variants diverge in applied optimizations. On the HIP backend, *MASS3DEA* shows a clear example: RAJA applies 24 additional loop-unroll transformations at O2 and O3, coinciding with the performance degradation observed in Section IV-B. In contrast, several Polybench kernels exhibit differences in SLP vectorization without corresponding changes in runtime behavior.

On OpenMP, *INIT_VIEW1D* exhibits a contrasting pattern. RAJA undergoes one fewer loop-unroll transformation at O2 and O3, coinciding with a performance degradation (approximately 42%). A related kernel, *INIT_VIEW1D_OFFSET*, does not show this difference and exhibits a smaller degradation (approximately 7%), isolating loop unrolling as a distinguishing factor between the pair.

Our observations highlight the role of compiler remarks as a diagnostic signal. Differences in loop unrolling align with performance divergence across both backends, whereas loop vectorization shows no observable differences and SLP vectorization exhibits limited and inconsistent impact. Compiler remarks therefore provide a useful but partial view: they identify where optimization behavior differs, but do not, by themselves, fully characterize the resulting performance differences.

Finding 7: Compiler remarks are an early localization signal: they identify where Native and RAJA receive different transformations and become explanatory when those optimizations propagate into IR structure or backend resource usage.

B. Control-Flow Changes

Next, we examine how abstraction affects the control-flow structure of the optimized LLVM IR, as shown in Figure 5. At this diagnostic layer, we focus on three aspects of control-flow structure that directly influence downstream code quality: loop-versioning guards that control optimized loop variants, loop-invariant branches that persist inside loop bodies, and live SSA value counts that determine register demand at the backend.

On HIP, we identify four control-flow patterns. First, loop-versioning differences identify cases where one variant exposes more optimization opportunities. In *ADI*, Native generates more loop-versioning guards than RAJA at every optimization level. These guards duplicate a loop body into an optimized fast path and a conservative fallback, gated by a runtime alias or alignment check.

Second, loop-invariant branches left inside inner loops introduce per-iteration overhead. Across the Polybench kernels (*ADI*, *ATAX*, *GEMVER*, *MVT*), RAJA shows a consistent

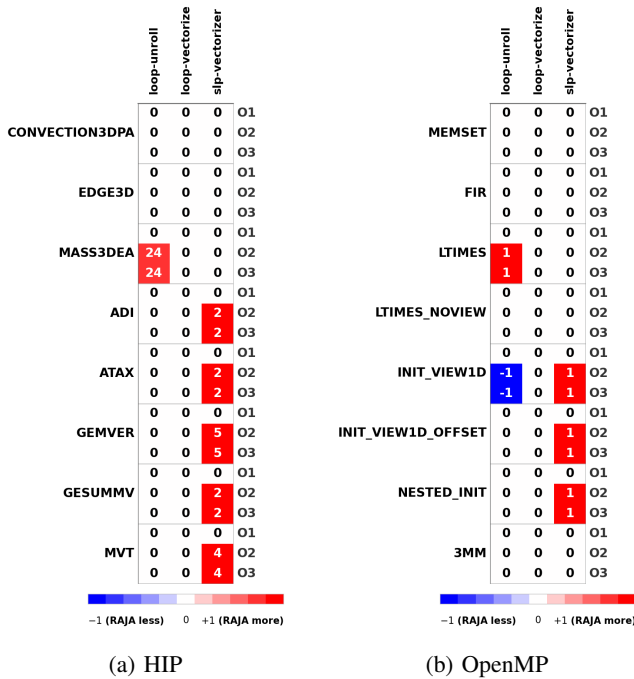
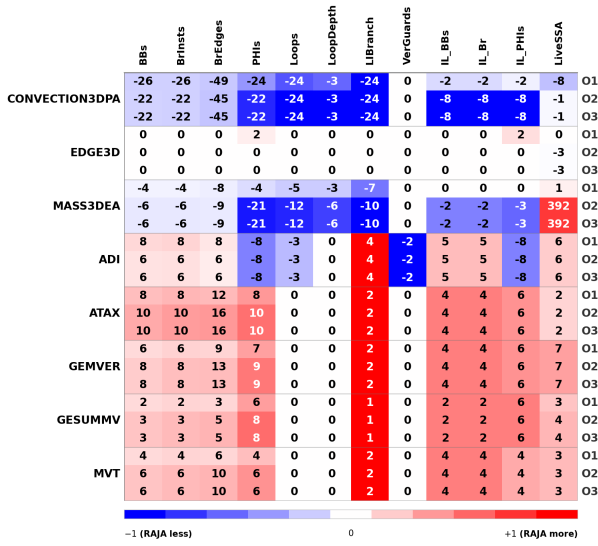
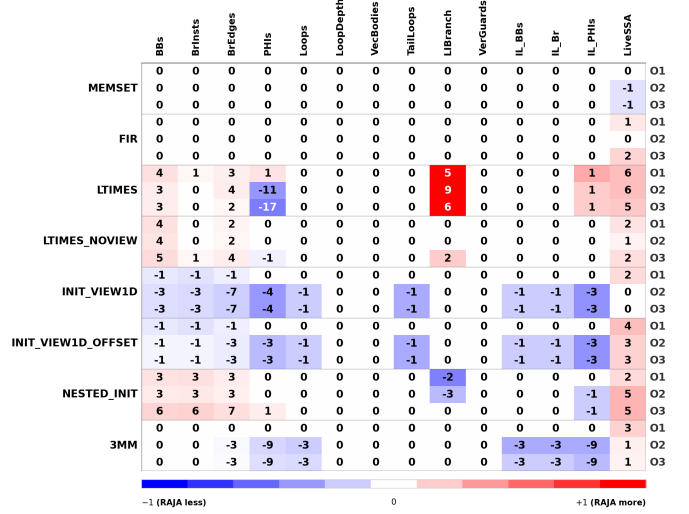


Fig. 4: Passed compiler remark differences between Native and RAJA for HIP (a) and OpenMP (b) backends. Red cells: RAJA has more passed remarks; blue: fewer; white: no difference.



(a) HIP



(b) OpenMP

Fig. 5: Differences in control-related IR metrics between Native and RAJA variants for the HIP (a) and OpenMP (b) backends. Red cells: RAJA has a higher metric value; blue: lower; white: no difference. Note that VecBodies and TailLoops are not available when using HIP.

increase, indicating that branches which could be hoisted remain inside the loop body. In *ADI*, this compounds with the missed loop versioning from the first pattern: fewer optimized loop paths *and* more per-iteration branch overhead together account for the large observed gap.

Third, simpler control flow can be beneficial. In *CONVECTION3DPA*, RAJA produces fewer basic blocks (-22), branch instructions (-22), and loops (-24) than Native across all optimization levels. Because loop-unroll remark counts are identical between variants (Section VI-A), this reduction reflects simpler loop structure not influenced by loop-unrolling.

Fourth, loop elimination from unrolling increases live-value pressure. In *MASS3DEA*, RAJA shows a decrease in loop count and an increase of $+392$ simultaneously live SSA values at O2 and O3. This pattern links directly to the loop unrolling identified in Section VI-A: unrolling expands the loop body, increasing the number of simultaneously live values and driving the VGPR pressure and spilling observed in Section V.

On OpenMP, control-flow differences are smaller and less consistent. *LTIMES* shows increased basic blocks ($+3$ to $+4$) and inner-loop branches at O2 and O3 but not at O1, aligning with its optimization-sensitive behavior. Its paired kernel *LTIMES_NOVIEW*, which removes *View* indirection, shows no such increase, isolating *View* as the source of the additional control flow. *INIT_VIEW1D* exhibits decreased inner-loop basic blocks and branches at O2 and O3, coinciding with a $\sim 42\%$ degradation; its pair *INIT_VIEW1D_OFFSET* shows a similar but smaller pattern with a correspondingly smaller gap ($\sim 7\%$). The remaining OpenMP kernels show negligible control-flow differences.

Finding 8: Control-flow metrics are diagnostic when they expose persistent structural overhead or downstream resource pressure. Loop-invariant branches indicate per-iteration overhead, reduced loop versioning identifies missed optimized paths, and unrolling-driven loop elimination links IR structure to backend register pressure.

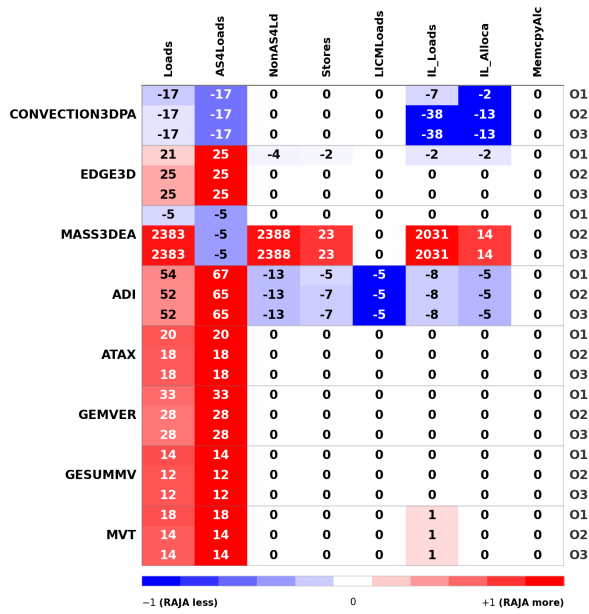
C. Memory-Access Changes

Memory IR metrics, as shown in Figure 6, capture how data access is represented in the IR. On HIP, we find that RAJA uses constant memory (AS4) loads for dispatch-related bookkeeping, such as lambda captures and kernel arguments. We therefore distinguish AS4 from non-AS4 loads to separate abstraction-related state access from the memory operations associated with the kernel computation itself. OpenMP does not expose this distinction, so all memory access must be interpreted uniformly.

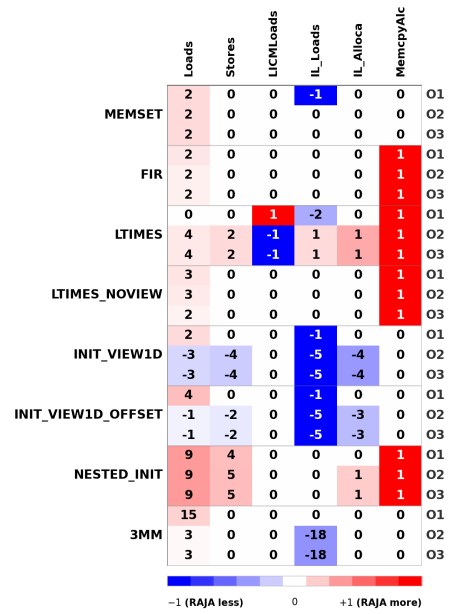
We observe three patterns.

First, load increases on HIP often reflect abstraction overhead rather than algorithmic changes. Across the Polybench kernels (*ATAX*, *GEMVER*, *GESUMMV*, *MVT*), RAJA’s higher total load counts are concentrated in AS4 loads; non-AS4 loads remain largely unchanged. On OpenMP, small load-count increases appear without corresponding performance sensitivity.

Second, memory-access changes can signal resource pressure. In *MASS3DEA* on HIP, RAJA shows $+2388$ non-AS4 loads and $+2031$ inner-loop loads at O2/O3—part of the same loop-body expansion that drives live-value pressure and scratch spilling (Sections VI-A and VI-B). On OpenMP,



(a) HIP



(b) OpenMP

Fig. 6: Differences in memory-related IR metrics between Native and RAJA for the HIP (a) and OpenMP (b) backends. Red cells: RAJA has a higher metric value; blue: lower; white: no difference. Note that AS4Loads and NonAS4Ld are not available when using OpenMP.

NESTED_INIT exhibits a similar pattern: increased memory operations accompany optimization-sensitive degradation.

Third, memory metrics require control-flow context. In *ADI* on HIP, load-count differences are ambiguous because the variants differ in loop versioning (Section VI-B): additional versioned paths duplicate memory operations in the IR, so aggregate counts may reflect static path duplication rather than hot-path behavior.

Finding 9: Memory metrics are diagnostic with address-space and control-flow context. RAJA load increases often reflect dispatch state rather than algorithmic memory changes, and become relevant when they align with loop-body expansion or resource pressure changes.

D. Compute-Operation Changes

Compute IR metrics (Figure 7) quantify the arithmetic and address-computation work in optimized IR. We observe two patterns. First, compute increases are diagnostic when they reflect unrolling-driven loop-body expansion. In *MASS3DEA* on HIP, RAJA shows +8,658 total and +8,769 inner-loop floating-point operations at O2/O3, together with fewer loops and +392 live SSA values. This pattern arises because unrolling replicates work from multiple iterations into a larger straight-line loop body, increasing both the number of floating-point instructions and the number of values simultaneously live in the IR. The same expansion corresponds to the VGPR pressure and scratch spilling identified in Section V. On OpenMP,

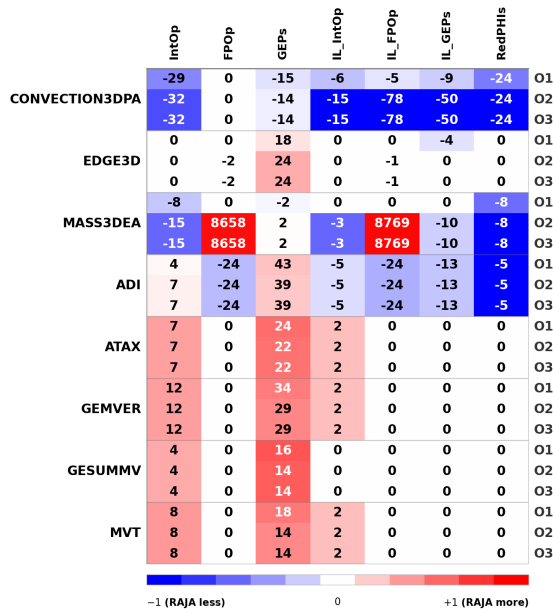
3MM and *NESTED_INIT* show a similar pattern: higher compute counts at O2/O3 correspond with optimization-sensitive runtime changes.

Second, modest increases in compute instructions do not consistently provide insight into performance differences. Across the Polybench kernels on HIP, RAJA shows small increases in integer arithmetic and GEP instructions. *GESUMMV* is the clearest example, with higher integer and GEP counts but no practically significant runtime gap. On OpenMP, kernels such as *FIR* exhibit minimal differences in compute, despite measurable performance variation.

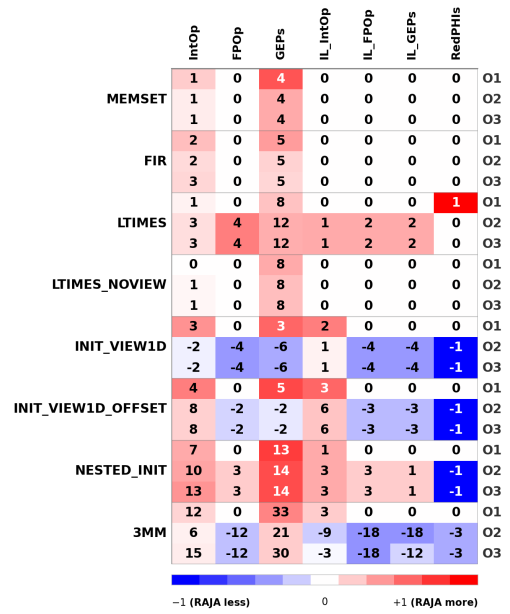
Finding 10: Compute metrics align with performance divergence when driven by loop-body expansion; small standalone increases in integer arithmetic and address computations do not consistently characterize runtime differences.

E. Reflection on Middle-End Analysis

Middle-end analysis is most informative when several IR metric categories form a consistent cross-layer pattern rather than being interpreted in isolation. On HIP, *MASS3DEA* is the clearest example: loop-unrolling remarks, reduced loop counts, higher LiveSSA, more loads, and floating-point expansion all indicate loop-body expansion, which increases register demand, reduces occupancy, and triggers scratch spilling. This agreement across control-flow, memory, compute, and backend-resource evidence makes the diagnosis interpretable. For the Polybench kernels, no single IR metric is decisive;



(a) HIP



(b) OpenMP

Fig. 7: Differences in compute-related IR metrics between the Native and RAJA variants for the HIP backend and OpenMP backend. Red cells indicate that RAJA has a higher value for the metric; blue cells indicate a lower value; white cells indicate no difference.

instead, many small differences across control-flow, memory, and compute metrics suggest accumulated abstraction overhead. However, *GESUMMV* shows that similar IR differences can still be practically benign, so IR divergence alone does not predict runtime impact. On OpenMP, middle-end signals are weaker: some kernels show runtime differences with limited IR changes, implying that part of the CPU-side behavior arises below or outside the LLVM middle end. Thus, middle-end analysis is most useful when it aligns with backend or structural evidence, and less informative when such alignment is absent.

Finding 11: Middle-end analysis is most reliable when multiple metric categories form a consistent cross-layer pattern; single-metric divergences are inconclusive; and on OpenMP, middle-end signals are weaker than on HIP.

Mystery 3: Middle-end analysis identifies which structural patterns align with performance overhead, but does not explain when and why abstractions at the source level produce those patterns. Under what source-level conditions do abstractions cause IR control-flow overhead, and can this be characterized statically before profiling?

VII. STATIC ANALYSIS FOR KERNEL OPTIMIZATION

A. Control-Flow Simplification

We address Mystery 3 directly through two optimization case studies. In the first, we focus on source-level restructuring.

On the HIP backend, four Polybench kernels (i.e., *ADI*, *ATAX*, *GEMVER*, and *MVT*) exhibit consistent performance degradation for RAJA relative to Native across all optimization levels. Our middle-end analysis identifies a recurring pattern: increased control-flow complexity driven primarily by loop-invariant branches within inner loops, with consequences for memory and compute characteristics. Moreover, in *ADI*, loop versioning amplifies this effect further: Native introduces additional control-flow paths that enable more aggressive optimization, while RAJA does not exploit these paths as effectively. Together, these signals identify control-flow structure as a primary contributor to the observed performance gap.

To trace this behavior to its source, we examine the variants at the source-code level. We find that, across all four kernels, the RAJA variants distribute their computational bodies across multiple lambdas, each introducing its own execution constraints. This structure maps directly to more complex control flow observed in the generated IR, producing the loop-invariant branch patterns identified in Section VI-B. We find that a single lambda can express the same computation equivalently, eliminating the nested dispatch structure and simplifying the resulting control flow.

Guided by this insight, we reimplement each kernel using a single lambda and recompile under identical conditions. Figure 8 compares mean runtime for Native, the original RAJA variant, and the modified RAJA variant. The single-lambda versions show consistent performance improvements across all four kernels, reducing or eliminating the performance gap. This confirms that multi-lambda structure introduces

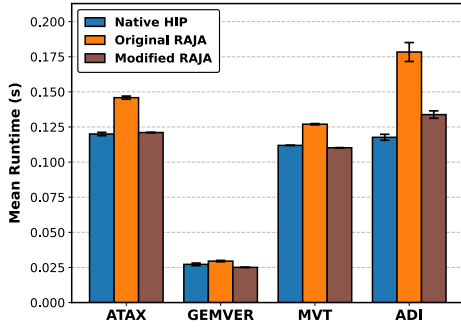


Fig. 8: Performance of Native, Original RAJA, and Modified RAJA for *ADI*, *ATAX*, *GEMVER*, and *MVT* on HIP at O1. The Modified RAJA removes or greatly reduces the runtime gap with Native for all kernels, confirming that nested lambdas caused the IR control-flow overhead.

measurable control-flow overhead in the IR, and that targeted source-level restructuring mitigates it. This case study demonstrates that our multi-layer diagnostic methodology is not only descriptive but prescriptive: patterns identified through static analysis translate directly into targeted source-level optimizations that recover performance, establishing a clear approach from diagnosis to optimization.

B. Mitigating Loop Unrolling

In the second case study, we address Mystery 3 through compiler-directed tuning rather than source-level restructuring. In our earlier analysis, *MASS3DEA* emerged as an optimization-sensitive kernel on the HIP backend, showing minimal performance difference at O1 but significant degradation at O2 and O3. Section V linked this degradation to increased register pressure in the RAJA variant: RAJA reaches the 256-VGPR architectural limit at higher optimization levels, spills first into AGPRs and then into scratch memory, and loses occupancy as a result. These hardware-level effects directly reduce execution efficiency.

Tracing this pressure upstream to the LLVM middle-end (Section VI) reveals the underlying cause. At O2 and O3, the compiler applies 24 additional loop-unroll optimizations to the RAJA variant. These optimizations reduce loop count and produce a large increase in live SSA values (+392), expanding the set of simultaneously live values and driving the register demand observed at the backend. This establishes a clear cross-layer causal chain: loop unrolling in the middle-end increases IR complexity and live-value pressure, which translates into higher register demand, spilling, and degraded runtime performance.

We target loop unrolling directly. We modify the RAJA variant of *MASS3DEA* by inserting *clang* pragma directives to disable loop unrolling on the inner computation, leaving all other compilation settings unchanged. This change limits loop-body expansion, reducing live-value pressure and mitigating excessive register usage. Figure 9 compares the original and modified RAJA variants with Native at O2 and O3. The

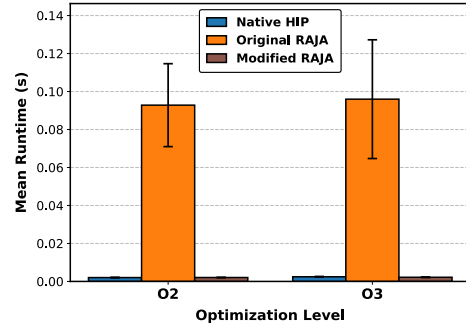


Fig. 9: Performance comparison of Native, Original RAJA, and Modified RAJA for *MASS3DEA* on HIP at O2 and O3. Disabling inner-loop unrolling removes or greatly reduces the runtime gap with Native, confirming that excessive unrolling caused register pressure and scratch spilling.

modified version recovers most or all of the performance gap, confirming that excessive loop unrolling caused the degradation. This shows that controlling compiler optimizations can be as effective as restructuring source-level abstractions.

The two case studies reinforce the prescriptive power of the methodology. By connecting compiler remarks, IR structure, and backend resource behavior, we identify not only the source of performance divergence but also the appropriate intervention. When abstraction structure drives overhead, as in the Polybench kernels, restructuring the lambda layout eliminates it. When compiler optimization behavior drives overhead, as in *MASS3DEA*, pragma-level tuning eliminates it without touching the abstraction itself. Both paths lead from diagnosis to optimization.

VIII. RELATED WORK

Performance portability frameworks. RAJA [1], [13] and Kokkos [14], [15] are widely used C++ abstractions for performance portability in HPC applications. RAJA encapsulates backend-specific execution policies behind loop-based C++ abstractions, allowing a single source implementation to target HIP, CUDA, OpenMP, and other models. Kokkos similarly abstracts parallel execution and memory access patterns across heterogeneous architectures. These frameworks define the programming model context for this work: they rely on the compiler to remove abstraction overhead and generate efficient backend code.

Performance portability evaluation. Prior work empirically evaluates performance portability across programming models, architectures, and applications. Pennycook et al. define metrics for performance portability [16], and Davis et al. compare CUDA, HIP, Kokkos, RAJA, OpenMP, OpenACC, and SYCL on leadership-class GPU systems [2]. These studies show that programming model choice can strongly affect performance but focus on runtime results. Our work complements them by diagnosing why Native and RAJA implementations diverge, using compiler-visible evidence rather than runtime measurements alone.

Benchmarking and performance-analysis tools. The RAJA Performance Suite [3], [17] provides the benchmark suite used in this study, with paired Native and RAJA variants of 55 loop-based kernels. Caliper [8] supplies runtime measurement and annotation to collect per-region performance data, and Thicket [9] supports multi-run, multi-configuration analysis. Our work extends these tools by associating runtime regions with function-level compiler information, enabling direct comparison of runtime behavior with static compiler structure.

Compiler infrastructure, optimization analysis, and IR characterization. LLVM provides the compiler infrastructure for middle-end analysis [18], and standard compiler literature describes the loop transformations, dependence analysis, and optimization techniques underlying loop versioning, unrolling, and vectorization [19], [20]. Prior work has also used LLVM IR as a static program representation for optimization and performance analysis. Autophase [21] and follow-up work use static IR features to learn optimization heuristics, while ProGraML [22] represents IR as graphs for program analysis. Our work uses LLVM IR differently: not to learn compiler decisions, but to diagnose when abstraction-induced structural differences influence observed runtime performance.

GPU resource usage and performance modeling. GPU performance is influenced by resource allocation decisions, including register usage, occupancy, and spilling. Prior work shows that occupancy alone does not predict performance; register pressure, instruction-level parallelism, and memory hierarchy effects must be considered together [23]. Performance modeling similarly links kernel structure and resource usage to performance [24]. Our work differs by tracing these backend effects to abstraction-induced differences in LLVM IR and compiler optimizations.

IX. CONCLUSIONS

We present a multi-layer workflow for diagnosing abstraction-induced performance differences in performance-portable applications. The workflow combines runtime measurements with compiler-visible evidence, including GPU backend resource usage, compiler optimization remarks, and LLVM IR metrics, to systematically identify the compiler behaviors responsible for performance divergence. We evaluate the workflow using 55 kernels from the RAJA Performance Suite across two backends (HIP and OpenMP), three optimization levels, and 1,980 experimental configurations.

Our results show that performance-portable abstractions generally achieve performance comparable to native implementations. Runtime differences exceeding $\pm 5\%$ occur in fewer than 9 of the 55 kernels on HIP and 19 kernels on OpenMP, while the majority of kernels remain within $\pm 2\%$. For the small subset of kernels exhibiting meaningful performance differences, runtime measurements alone are insufficient to explain the observed behavior. Reliable diagnosis requires correlating runtime observations with complementary compiler-visible evidence spanning backend resource usage, compiler optimization remarks, and LLVM IR structure.

Finally, we demonstrate that the proposed workflow guides effective optimization. The compiler diagnostics identify source-level restructuring opportunities that recover Native-level performance for four Polybench kernels and expose an optimization-sensitive case (*MASS3DEA*) in which disabling loop unrolling eliminates up to 3,220 bytes of scratch spilling and restores Native-level performance at O2 and O3. These results show that integrating runtime measurements with compiler-visible evidence provides a systematic approach for understanding, diagnosing, and improving performance portability in modern portability frameworks.

ACKNOWLEDGEMENTS

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344 (LLNL-CONF-2018937). Generative AI tools were used to correct grammatical errors and to assist with aesthetic modifications to plotting code used to produce figures. All analysis, data interpretation, validation, and final manuscript content were performed and reviewed by the authors.

REFERENCES

- [1] R. D. Hornung and J. A. Keasler, “The RAJA Portability Layer: Overview and Status,” Lawrence Livermore National Lab. (LLNL), Livermore, CA (United States), Tech. Rep. LLNL-TR-661403, Sep. 2014. [Online]. Available: <https://www.osti.gov/biblio/1169830>
- [2] J. H. Davis, P. Sivaraman, J. Kitson, K. Parasyris, H. Menon, I. Minn, G. Georgakoudis, and A. Bhatle, “Taking GPU Programming Models to Task for Performance Portability,” in *Proceedings of the 39th ACM International Conference on Supercomputing*, ser. ICS ’25. New York, NY, USA: Association for Computing Machinery, Aug. 2025, pp. 776–791. [Online]. Available: <https://dl.acm.org/doi/10.1145/3721145.3730423>
- [3] R. D. Hornung and H. E. Jones, “RAJA Performance Suite,” Sep. 2017, computer software. [Online]. Available: <https://doi.org/10.11578/dc.20201001.36>
- [4] O. Pearce, J. Burmark, R. Hornung, B. Bogale, I. Lumsden, M. McKinsey, D. Yokelson, D. Boehme, S. Brink, M. Tauber, and T. Scogland, “RAJA Performance Suite: Performance Portability Analysis with Caliper and Thicket,” in *2024 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*. IEEE, Nov. 2024, pp. 1206–1218.
- [5] Advanced Micro Devices, Inc., “HIP Programming Guide,” 2023, accessed: 2026. [Online]. Available: <https://rocm.docs.amd.com/projects/HIP/en/latest/>
- [6] L. Dagum and R. Menon, “OpenMP: An Industry Standard API for Shared-Memory Programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [7] Advanced Micro Devices, Inc., “ROCm Documentation,” 2023, accessed: 2026. [Online]. Available: <https://rocm.docs.amd.com/>
- [8] D. Boehme, T. Gambelin, D. Beckingsale, P.-T. Bremer, A. Gimenez, M. LeGendre, O. Pearce, and M. Schulz, “Caliper: Performance Introspection for HPC Software Stacks,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’16. IEEE Press, 2016.
- [9] S. Brink, M. McKinsey, D. Boehme, C. Scully-Allison, I. Lumsden, D. Hawkins, T. Burgess, V. Lama, J. Lüttgau, K. E. Isaacs, M. Tauber, and O. Pearce, “Thicket: Seeing the Performance Experiment Forest for the Individual Run Trees,” in *32nd Intl Symposium on High-Performance Parallel and Distributed Computing*, June 2023.
- [10] C. Lattner, “LLVM: An Infrastructure for Multi-Stage Optimization,” Master’s thesis, University of Illinois at Urbana-Champaign, December 2002. [Online]. Available: <https://llvm.org/pubs/2002-12-LattnerMSThesis.html>

- [11] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [12] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: <https://www.jstor.org/stable/4615733>
- [13] D. A. Beckingsale, J. Burmark, R. Hornung, H. Jones, W. Killian, A. J. Kunen, O. Pearce, P. Robinson, B. S. Ryuji, and T. R. Scogland, "RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, Nov. 2019, pp. 71–81. [Online]. Available: <https://ieeexplore.ieee.org/document/8945721>
- [14] H. Carter Edwards, C. R. Trott, and D. Sunderland, "Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, vol. 74, no. 12, pp. 3202–3216, 2014, domain-Specific Languages and High-Level Frameworks for High-Performance Computing. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731514001257>
- [15] H. C. Edwards and C. R. Trott, "Kokkos: Enabling Performance Portability Across Manycore Architectures," in *2013 Extreme Scaling Workshop (xsw 2013)*, Aug. 2013, pp. 18–24, iSSN: 2381-2036. [Online]. Available: <https://ieeexplore.ieee.org/document/6805038>
- [16] S. J. Pennycook, J. D. Sewall, and V. W. Lee, "A metric for performance portability," *CoRR*, vol. abs/1611.07409, 2016. [Online]. Available: <http://arxiv.org/abs/1611.07409>
- [17] D. Yokelson, S. Brink, J. Burmark, M. McKinsey, B. Bogale, I. Lumsden, M. Taufer, T. Scogland, and O. Pearce, "Cross-Architecture Performance Analysis Using the RAJA Performance Suite," in *Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 617–626. [Online]. Available: <https://doi.org/10.1145/3754598.3754668>
- [18] C. Lattner and V. Adve, "LLVM: a compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, Mar. 2004, pp. 75–86. [Online]. Available: <https://ieeexplore.ieee.org/document/1281665>
- [19] R. Allen and K. Kennedy, *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. Morgan Kaufmann, 2001.
- [20] S. S. Muchnick, *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [21] C. Cummins, P. Petoumenos, Z. Wang, and H. Leather, "End-to-end deep learning of optimization heuristics," in *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2017, pp. 219–232.
- [22] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoeffler, M. F. P. O'Boyle, and H. Leather, "ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 2244–2253. [Online]. Available: <https://proceedings.mlr.press/v139/cummins21a.html>
- [23] V. Volkov, "Better Performance at Lower Occupancy," in *GPU Technology Conference*, 2010.
- [24] S. S. Baghsorkhi, M. Delahaye, S. J. Patel, W. D. Gropp, and W.-m. W. Hwu, "An Adaptive Performance Modeling Tool for GPU Architectures," in *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 105–114. [Online]. Available: <https://doi.org/10.1145/1693453.1693470>

APPENDIX A
ARTIFACT DESCRIPTION APPENDIX: A MULTI-LAYER
WORKFLOW FOR DIAGNOSING PERFORMANCE
PORTABILITY IN RAJA APPLICATIONS

A. Abstract

This description explains how to reproduce the experiments of the paper, which compares Native and RAJA implementations of 55 RAJA Performance Suite (RAJAPerf) kernels across the HIP and OpenMP backends, three problem sizes, and three optimization levels (1,980 configurations, 59,400 kernel executions). Reproducing the results from scratch means rebuilding RAJAPerf at each optimization level, running the sweep on AMD MI300A hardware, collecting static compiler metrics with two custom LLVM pass plugins, merging both with Thicket, and recomputing the statistics behind each figure. Each step is specified below, and Section A-E gives concrete values to check against.

B. Description

1) Check-list (artifact meta information):

- **Algorithm:** 55 RAJAPerf kernels, Native and RAJA variants
- **Program:** RAJAPerf (C++); 2 LLVM pass plugins; Jupyter
- **Compilation:** hipcc/amdclang++ 19.0.0 (ROCm 6.4.0), -O1/-O2/-O3
- **Binary:** raja-perf.exe, one per opt level/build
- **Data set:** RAJAPerf-generated; 8M, 32M, 128M elements
- **Run-time environment:** TOSS 4, ROCm 6.4.0, cray-mpich 8.1.32, Flux, Spack
- **Hardware:** 1 node of LLNL Tuolumne, 4× AMD MI300A (gfx942)
- **Run-time state:** exclusive node; HIP: 4 ranks, 1 GPU each; OpenMP: 1 rank, 84 cores
- **Execution:** Flux batch, 30 repetitions per configuration
- **Output:** Caliper .cali profiles; JSON static metrics; JSONL region maps; figures
- **Experiment workflow:** build → sweep → collect metrics → merge → statistics and figures
- **Experiment customization:** opt level, size, kernel set, backend, tuning, significance thresholds
- **Publicly available?:** RAJA, RAJAPerf, Caliper, Thicket are open source; the plugins, scripts, and collected data will be publicly available with the publication artifact

2) *How software can be obtained:* The components carrying the experiment are public: RAJAPerf (<https://github.com/LLNL/RAJAPerf>) supplies the paired Native and RAJA kernel variants, Caliper (<https://github.com/LLNL/Caliper>) the run-time annotation and profiles, and Thicket (<https://github.com/LLNL/thicket>) the multi-configuration analysis. RAJAPerf is unmodified except for the two case studies of Sec. VII. The two pass plugins are developed in benchpark-llvm (<https://github.com/LLNL/benchpark-llvm>), presently private and slated to become public; the scripts, notebook, and collected data will be released with the publication artifact, with the data under a citable DOI. So that reproduction does not depend on that release, the sections below give each plugin's flags, the data it emits, and the statistics computed from it in enough detail to rebuild the pipeline and regenerate the dataset from source.

3) *Hardware dependencies:* An AMD MI300A (gfx942) node is required: Kernel Resource Usage (KRU) metrics come only from the AMD GPU backend, and the register and occupancy thresholds are architecture specific. Each Tuolumne node holds four MI300A APU's (CDNA 3 GPU plus Zen 4 cores, unified HBM3). HIP runs use all four (4 ranks, 1 GPU each); OpenMP runs use one rank with 84 cores on the same node, so both backends run on the same physical node.

4) *Software dependencies:* ROCm 6.4.0, cray-mpich 8.1.32, CMake, Ninja, Flux, and a Spack environment providing Caliper and Adiak. Analysis needs Python 3 with thicket, hatchet, pandas, numpy, scipy, statsmodels, matplotlib, and jupyter.

5) *Datasets:* No input dataset is needed: RAJAPerf generates its own arrays from --size. The workflow instead produces about 27 GB. The sweep writes one Caliper .cali profile per variant, size, optimization level, repetition, and tuning group; each extraction build writes one JSON file per translation unit, split into host and device records, holding its KRU values, remarks, and IR metrics; the association runs write one JSONL trace log per variant. Merging these into Thicket objects yields roughly 350 MB of pickle files, which the analysis reads and which will be included in the data release.

C. Installation

Set up the environment:

```
module load rocm/6.4.0 cmake ninja \
    cray-mpich/8.1.32
spack env activate \
    $REPO/spack_environments/tuo-env
export ROCM=/opt/rocm-6.4.0
export LLVM=$ROCM/lib/llvm
```

Three build flavors are needed. (1) **Runtime builds**, one per optimization level, which produce every timing number. The optimization flag is overridden on both the C++ and HIP compile lines so that all other settings are held constant:

```
OPT=-O2
F="-fPIC $OPT -g"
cmake -B build/raw$OPT -G Ninja \
    -DCMAKE_BUILD_TYPE=RelWithDebInfo \
    -DCMAKE_CXX_COMPILER=$ROCM/bin/hipcc \
    -DCMAKE_CXX_COMPILER=$LLVM/bin/clang++ \
    -DCMAKE_CXX_ARCHITECTURES=gfx942 \
    -DCMAKE_PREFIX_PATH=$ROCM/lib/cmake \
    -DENABLE_HIP=On -DENABLE_OPENMP=On \
    -DENABLE_MPI=On -DENABLE_TESTS=Off \
    -DBLT_CXX_STD=c++17 \
    -Dcaliper_DIR=$CALI/share/cmake/caliper \
    -DCMAKE_CXX_FLAGS_RELWITHDEBINFO="$F" \
    -DCMAKE_HIP_FLAGS_RELWITHDEBINFO="$F"
ninja -C build/raw$OPT
```

(2) **Extraction builds**, one per optimization level, identical except for the added plugin and remark flags. Compiling emits the KRU, remark, and IR-metric JSON files; the plugin only observes the pipeline, so codegen matches the runtime build:

```
F="-fPIC $OPT -g \
    -fpass-plugin=$EXT/ExtractionPlugin.so \
    -mllvm -pass-remarks=.* \
    -mllvm \
    -pass-remarks-analysis=kernel-resource-usage"
cmake -B build/ext$OPT ... \
    -DCMAKE_CXX_FLAGS_RELWITHDEBINFO="$F" \
    -DCMAKE_HIP_FLAGS_RELWITHDEBINFO="$F"
```

(3) **One association build** at $-O2$, used only to map Caliper regions to compiled functions. It instruments the pipeline and is never used for timing:

```
F="-fPIC -O2 -g -S -emit-llvm \
-fpass-plugin=$ASC/CaliperAssociationPlugin.so"
LD="-L$ASC -lcali_runtime -ldl \
-Wl,--export-dynamic"
cmake -B build/assoc ... \
-DCMAKE_CXX_FLAGS_RELWITHDEBINFO="$F" \
-DCMAKE_HIP_FLAGS_RELWITHDEBINFO="$F" \
-DCMAKE_EXE_LINKER_FLAGS="$LD"
```

D. Experiment workflow

1) *Runtime sweep*: One Flux job per (opt level, repetition, tuning group); kernels are grouped by launch tuning (block_256, block_64, block_25, launch_256) so each runs under its intended configuration. `--size` is per rank, so 8388608 over 4 ranks is the 32M point:

```
flux run --nodes=1 --exclusive --ntasks=4 \
--gpus-per-task=1 \
build/raw-O2/bin/raja-perf.exe \
--variants RAJA_HIP --tunings block_256 \
--size 8388608 --kernels $KERNELS \
--warmup-kernels --outdir $OUT
```

Loop this over the four variants (Base_HIP, RAJA_HIP, Base_OpenMP, RAJA_OpenMP), the three optimization levels, the tuning groups, and 30 repetitions, writing each run to its own `--outdir`; that is the 1,980-configuration sweep. For HIP, use per-rank sizes 2097152, 8388608, and 33554432 across four ranks to realize the 8M, 32M, and 128M aggregate problem sizes. For OpenMP, use `--ntasks=1`, no `--gpus-per-task`, and sizes 8388608, 33554432, and 134217728 to realize the same problem sizes. Only *LTIMES* and *LTIMES_NOVIEW* use `launch_256`; *CONVECTION3DPA*, *DIFFUSION3DPA*, and *MASS3DEA* use `block_64`, *MASS3DPA* uses `block_25`, and the rest use `block_256`.

2) *Static metrics*: Building the extraction flavor already writes the JSON metric files. The region-to-function map is collected once per variant with a single check run:

```
export LD_LIBRARY_PATH=\
$ASC:$ROCM/llvm/lib:$LD_LIBRARY_PATH
build/assoc/bin/raja-perf.exe -v RAJA_HIP \
-t block_256 --size 128 --kernels $KERNELS \
--checkrun 1
```

Repeat per variant and tuning group, concatenating the emitted `trace_log.jsonl` files; one check run per group is enough, since the mapping is static.

3) *Analysis*: Load the profiles with `Thicket.from_caliperreader`, one `Thicket` per variant and problem size with optimization level as a column axis, and join the plugin JSON to the matching kernel regions through the association map. Then, per kernel and configuration: take the mean, median, and standard deviation of Avg time/rank over the 30 repetitions; test Native against RAJA with a two-sided Mann–Whitney U test; apply Holm–Bonferroni correction jointly to the 990 Native–RAJA hypothesis tests; and record the percent change $(RAJA - Native)/Native$. For the static metrics take the per-kernel maximum and record both the raw RAJA-minus-Native difference and the symmetric normalized difference

$(R - N)/(R + N)$ used for cell color. The figures follow: significance heatmaps over all kernels and sizes, a percent-change heatmap at 32M for kernels significant at any level, and per-kernel grids for KRU, remarks, and the IR groups.

E. Evaluation and expected result

Static metrics are deterministic for a fixed compiler and build configuration, so on ROCm 6.4.0 they should match the paper exactly and are the strongest check that the pipeline was rebuilt correctly. The clearest single value to verify is the *MASS3DEA* KRU row of the backend-resource grid (HIP, 32M):

		Native	RAJA
VGPRs	O2/O3	183	256
Scratch(B)	O2	0	2888
Scratch(B)	O3	0	3220
VGPR spill	O2	0	895
VGPR spill	O3	0	988
Occupancy	O2/O3	2	1

The corresponding middle-end signal is +24 loop-unroll remarks and +392 live SSA values for RAJA at $-O2/-O3$, and *CONVECTION3DPA* should show -22 basic blocks, -22 branch instructions, and -24 loops for RAJA at every level.

Runtime results are statistical. Expect fewer than 9 of 55 kernels on HIP and 19 on OpenMP to exceed a $\pm 5\%$ Native–RAJA difference at 32M, *MASS3DEA* to be the extreme HIP outlier at $-O2$ and $-O3$, and the case-study bar charts to show the modified variants closing most of the gap. Re-running on different hardware will change absolute runtimes and may flip borderline significance decisions; the structural conclusions should not change.

F. Experiment customization

Optimization levels, problem size, repetition count, kernel list, and tuning groups are all command-line arguments to `raja-perf.exe`, so the sweep can be widened or narrowed without touching source. On the analysis side: which kernels enter the static-metric grids (the paper uses the eight HIP and eight OpenMP kernels surviving runtime filtering), which columns are plotted, the $\pm 5\%$ threshold, and the $p < 0.05$ level. The case-study variants are built by editing RAJAPerf sources: a `clang` unroll-disabling pragma on the *MASS3DEA* inner loop, and a single-lambda rewrite of *ADI*, *ATAX*, *GEMVER*, and *MVT*. Both are then rebuilt and run unchanged. The workflow generalizes to any abstraction offering paired equivalent variants under an LLVM toolchain.

G. Notes

Keeping the association plugin out of the timing path is deliberate: it instruments compilation and can perturb both compiler and runtime behavior, so it contributes only the region-to-function map. KRU is HIP-only, so the OpenMP analysis rests on remarks and IR metrics alone, a weaker signal. Runs used exclusive single-node allocations, and each configuration is repeated 30 times because the filtering step depends on the run-to-run distribution.